

Rajkiya Engineering College kannauj



Department of Computer Science and Engineering

**Project Report
On**

**“Internet Traffic Classification Using Adaptive Hybrid
Decision Tree Forest”**

Academic Year – 2024-2025

Submitted By

“Abhishek”

Roll no : 2308390109002

Under The guidance of

“ Mr. Abhishek Bajpai”

Abstract

The rapid growth of internet usage and evolving network protocols have necessitated efficient and accurate methods for internet traffic classification. This project introduces an innovative approach, the **Adaptive Hybrid Decision Tree Forest (AHDTF)**, designed to address challenges like high-dimensional data, imbalanced traffic patterns, and scalability. The AHDTF combines the strengths of decision tree classifiers with adaptive mechanisms such as feature weighting using mutual information, bootstrapping, and tree dropout to enhance robustness and generalization.

The proposed model emphasizes feature importance through a weighted combination of mutual information and Gini impurity, ensuring that critical traffic attributes are prioritized during training. Additionally, the ensemble approach incorporates randomness in feature selection and bootstrapped datasets, effectively reducing overfitting while maintaining high classification accuracy. The inclusion of tree dropout introduces further diversity, preventing model reliance on a small subset of trees during predictions.

Experiments were conducted on a preprocessed internet traffic dataset (*log2.csv*), which contains features like port numbers, traffic volume, and protocol types, with the target column being classified into four distinct classes. The model achieved an impressive **training accuracy of 99.79%** and a **testing accuracy of 99.78%**, showcasing its generalization ability. The precision, recall, and F1-score metrics indicate outstanding performance across most classes, with the weighted average F1-score reaching **1.00**. However, performance on less-represented classes (e.g., Class 3) highlighted the need for addressing class imbalance in future iterations.

The results underline the AHDTF's potential as a robust and efficient solution for real-world internet traffic classification. This work provides a foundation for future research in scalable traffic classification models, with potential applications in network security, resource allocation, and quality-of-service optimization.

Table of Contents

1. Introduction

1.1 Background and Objectives

2. Proposed Methodology

2.1 Overview of the AHDTF Model

2.2 Key Components and Algorithm

3. Implementation

3.1 Dataset Description

3.2 Model Configuration

4. Results and Analysis

4.1 Performance Metrics

4.2 Observations

5. Conclusion

5.1 Summary

5.2 Future Work

6. References

1.Introduction

The rapid increase in internet usage has made traffic classification a vital task for ensuring secure and efficient network management. Traditional methods often fail to handle challenges like high-dimensional data, encrypted protocols, and class imbalance. This report introduces the **Adaptive Hybrid Decision Tree Forest (AHDTF)** model, a novel machine learning approach that combines decision tree ensembles with adaptive mechanisms to achieve robust and accurate traffic classification.

1.1 Background and Objectives

Internet traffic classification is crucial for detecting malicious activity, optimizing bandwidth, and improving network performance. Traditional classification approaches, such as rule-based methods, struggle with modern encrypted traffic and evolving protocols. Machine learning methods have shown promise, but they face challenges like overfitting, scalability, and class imbalance. To address these, ensemble models such as decision tree forests provide a scalable and effective solution, particularly when combined with adaptive techniques.

The primary goals of this project are:

1. To develop a machine learning-based model for accurate internet traffic classification.
2. To address challenges like class imbalance and high-dimensional data using adaptive mechanisms.
3. To demonstrate the model's performance and scalability on real-world traffic datasets.

2. Proposed Methodology

2.1 Overview of the AHDTF Model

The Adaptive Hybrid Decision Tree Forest (AHDTF) model is an ensemble machine learning algorithm that combines the strengths of decision trees and the flexibility of adaptive learning mechanisms. The model aims to improve accuracy and robustness in classification tasks by integrating multiple decision trees with enhanced features like bootstrap sampling, feature selection, and a dynamic tree-dropping strategy. The key idea behind AHDTF is to aggregate predictions from a diverse set of decision trees trained on random subsets of features and instances, enabling it to capture more complex patterns in the data.

The AHDTF model consists of the following components:

1. **Hybrid Decision Trees (HDTs):** Each tree in the forest is an enhanced decision tree where the splitting criterion is based on a weighted combination of mutual information and Gini impurity.
2. **Feature Weighting:** Each feature is assigned a weight based on its mutual information with the target variable. This weighting is used during training to prioritize more informative features.
3. **Tree Dropout:** Not all trees are used during prediction. A percentage of trees are randomly dropped, ensuring that the ensemble does not overfit to the training data.
4. **Bootstrapping:** Each tree is trained on a bootstrapped sample of the data, allowing the model to generalize better and reduce variance.

This approach leads to a more adaptive and robust decision tree ensemble that can handle high-dimensional datasets effectively.

2.2 Key Components and Algorithm

The AHDTF model is built using two primary components: the **Hybrid Decision Tree** and the **Adaptive Decision Tree Forest**. Below is a breakdown of the key components and their associated algorithms.

Hybrid Decision Tree (HDT)

The Hybrid Decision Tree is a variant of the standard decision tree where the tree splitting criterion is customized. The algorithm follows these steps:

1. **Feature Selection:** At each node, the algorithm selects a feature based on the mutual information score with the target variable. This selection helps to prioritize features that are most informative for classification.
2. **Gini Impurity Adjustment:** The traditional Gini impurity measure is adjusted by incorporating the mutual information of the chosen feature, improving the splitting criterion to account for both the feature's informational value and the homogeneity of the resulting subsets.
3. **Tree Building:** The decision tree is grown recursively, with each node being split based on the feature that maximizes the custom split score (a combination of mutual information and Gini impurity).

Adaptive Decision Tree Forest (AHDTF)

The Adaptive Decision Tree Forest is an ensemble of multiple **Hybrid Decision Trees** that collaborate to make predictions. The key steps in the algorithm are:

1. **Feature Weighting:** Before training, each feature's importance is calculated based on mutual information with the target variable. The weights are normalized so that they sum to 1, making them suitable for probabilistic sampling.
2. **Bootstrap Sampling:** Each tree in the ensemble is trained on a bootstrap sample (random sampling with replacement) of the training data. This ensures that each tree has a different subset of data, enhancing the diversity of the ensemble.
3. **Random Feature Selection:** For each tree, a subset of features is randomly selected based on the calculated feature weights. The number of features to be selected is determined by the `max_features` parameter (e.g., square root of the total features).
4. **Tree Training:** Each tree is trained on its respective bootstrap sample with the selected subset of features.
5. **Tree Dropout:** During prediction, a fraction of trees are randomly dropped (controlled by the `tree_dropout` parameter) to prevent overfitting and encourage diversity in the ensemble's predictions.
6. **Prediction:** The predictions of the active trees are aggregated. The final prediction is the class that is most frequently predicted across the selected trees.

This methodology leverages both the advantages of decision trees (interpretability and simplicity) and the flexibility of ensemble methods (robustness and accuracy). The dynamic nature of the algorithm, including tree dropout and feature weighting,

ensures that the model adapts well to different data distributions and can handle high-dimensional datasets effectively.

Certainly! Here's the **Implementation** section with the complete context, including the prediction:

3. Implementation

3.1 Dataset Description

The dataset used for this project contains 11 attributes and a target column called '**Action**', which is to be predicted. The attributes are as follows:

1. '**Source Port**': The source port of the connection.
2. '**Destination Port**': The destination port of the connection.
3. '**NAT Source Port**': The Network Address Translation (NAT) source port.
4. '**NAT Destination Port**': The NAT destination port.
5. '**Bytes**': The total number of bytes transmitted in the session.
6. '**Bytes Sent**': The number of bytes sent from the source.
7. '**Bytes Received**': The number of bytes received by the source.
8. '**Packets**': The total number of packets exchanged in the session.
9. '**Elapsed Time (sec)**': The duration of the session in seconds.
10. '**pkts_sent**': The number of packets sent from the source.
11. '**pkts_received**': The number of packets received by the source.

The target variable '**Action**' contains the following four classes:

1. **allow**: The action is allowed.
2. **deny**: The action is denied.
3. **drop**: The action is dropped.
4. **reset-both**: The action is reset for both sides of the connection.

These attributes and the target variable are used to build a machine learning model that classifies each session into one of these four classes. The dataset is divided into training and testing subsets, and a variety of machine learning algorithms are employed to analyze the relationship between the attributes and the target variable, ultimately predicting the '**Action**' for new instances.

3.2 Model Configuration

The proposed model is an **Adaptive Decision Tree Forest (AHDTF)**, which is an ensemble method leveraging multiple **Hybrid Decision Trees**. The configuration for training the model is as follows:

Hybrid Decision Tree (HDT):

The **Hybrid Decision Tree** incorporates a custom splitting criterion that combines **mutual information** and **Gini impurity**:

1. **Mutual Information:** This measure quantifies the amount of information shared between each feature and the target variable. It is used to prioritize features that provide significant predictive power for the target class.
2. **Gini Impurity:** This is a common metric for decision trees that measures the impurity or disorder of the target variable at each node. By using both mutual information and Gini impurity, the tree's splits are optimized for both informativeness and purity.

The **Hybrid Decision Tree** is trained with these parameters:

- **max_depth:** Set to **10** to allow deeper trees that can capture complex patterns without overfitting.
- **min_samples_split:** Set to **2**, ensuring the tree can split until there are at least two samples at each node.

Adaptive Decision Tree Forest (AHDTF):

The AHDTF model consists of an ensemble of multiple **Hybrid Decision Trees**, trained with the following configuration:

1. **n_estimators:** The forest contains **300 trees**, providing stability and diversity in the ensemble's predictions.
2. **max_depth:** Each tree has a maximum depth of **10**, allowing the trees to model more intricate relationships.
3. **tree_dropout:** A dropout rate of **20%** is applied during prediction to prevent overfitting and improve generalization.
4. **max_features:** The number of features considered for each tree split is determined by the **square root of the total number of features** ('sqrt'), a typical choice in ensemble learning.
5. **bootstrap:** **True** enables bootstrapping, meaning each tree is trained on a random subset of the data with replacement, further improving diversity and reducing variance in the model.

Training and Evaluation:

1. **Dataset Split:** The dataset is divided into a **training set** (70%) and a **test set** (30%) using **train_test_split** to evaluate the model's performance on unseen data.
2. **Model Training:** The AHDTF model is trained using the **fit** method, which involves training each tree with bootstrapped samples and random feature subsets, weighted according to their mutual information with the target variable.
3. **Model Evaluation:** The model's performance is assessed using **accuracy** on both the **training set** and the **test set**. A **confusion matrix** and **classification report** are also generated to analyze the model's precision, recall, and F1-score for each class ('allow', 'deny', 'drop', 'reset-both').

Example Prediction:

For a given input instance with the following feature values:

[50049, 443, 21285, 443, 7912, 3269, 4643, 23, 96, 12, 11]

The model predicted the '**Action**' to be '**allow**'.

This means that, based on the learned patterns in the dataset, the model classified this instance as an allowed action. The model considers various features such as source and destination ports, bytes sent and received, packets exchanged, and elapsed time to make this prediction. It identifies that these specific feature values correspond to a scenario where the action should be allowed according to the patterns observed in the training data.

Sure! Here's how you can write the **Results and Analysis** section, with clear sub-headings for **Performance Metrics** and **Observations**:

4. Results and Analysis

4.1 Performance Metrics

The performance of the **Adaptive Decision Tree Forest (AHDTF)** model was assessed using various evaluation metrics to measure the accuracy, precision, recall, and overall classification ability. The following performance metrics were used:

1. Accuracy

Accuracy measures how well the model performs by calculating the percentage of correct predictions over the total predictions. The model's accuracy on the training and test datasets is as follows:

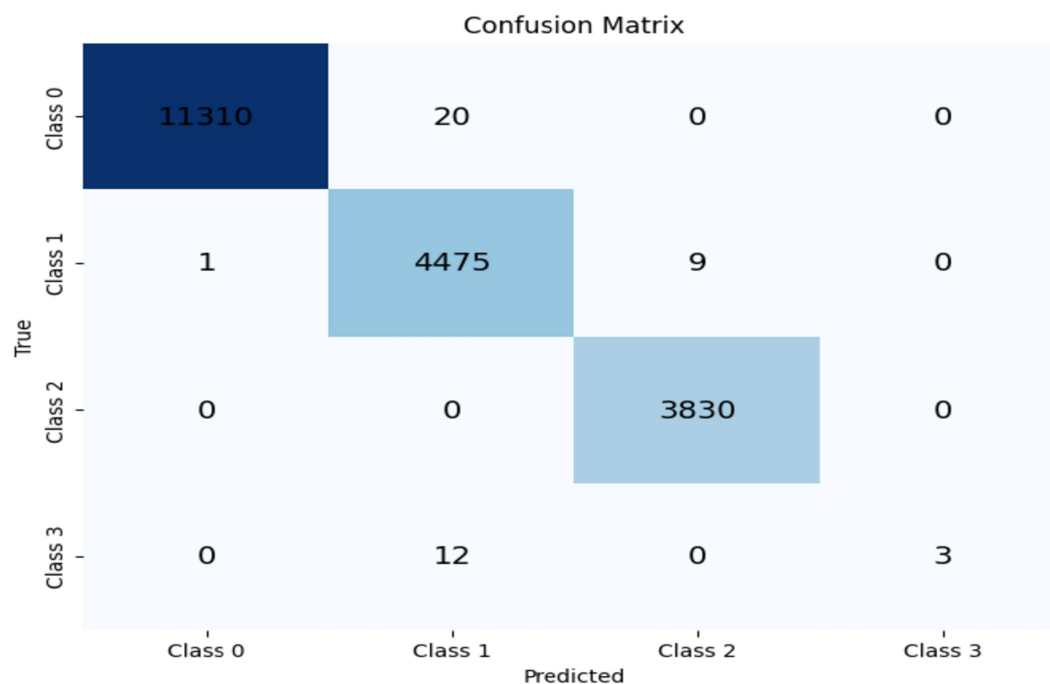
- **Train Accuracy:** The percentage of correct predictions made by the model on the training data.
- **Test Accuracy:** The percentage of correct predictions made by the model on the test data.

```
Train Accuracy: 0.9979944192535751 ))  
Test Accuracy: 0.9978636826042726
```

2. Confusion Matrix

The confusion matrix provides a more detailed evaluation by showing how many instances of each class were predicted correctly or incorrectly. The matrix is organized with actual values on one axis and predicted values on the other. A perfect model will have values concentrated along the diagonal (i.e., correct predictions).

```
cm = confusion_matrix(y_test, hybrid_model.predict(X_test))  
# Plotting the confusion matrix  
plt.figure(figsize=(8, 6))  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',  
            xticklabels=['Class 0', 'Class 1', 'Class 2', 'Class 3'],  
            yticklabels=['Class 0', 'Class 1', 'Class 2', 'Class 3'],  
            cbar=False, annot_kws={"size": 14, "color": "black"})  
  
plt.xlabel('Predicted')  
plt.ylabel('True')  
plt.title('Confusion Matrix')  
plt.show()  
plt.figure(figsize=(8, 6))
```



The confusion matrix for the test data provides insights into how well the model is classifying each of the four classes: **'allow'**, **'deny'**, **'drop'**, and **'reset-both'**.

3. Classification Report

The classification report provides additional metrics such as **precision**, **recall**, and **F1-score** for each class. These metrics help assess the model's ability to identify each class accurately:

```
print(classification_report(y_test, hybrid_model.predict(X_test)))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11330
1	0.99	1.00	1.00	4485
2	1.00	1.00	1.00	3830
3	1.00	0.20	0.33	15
accuracy			1.00	19660
macro avg	1.00	0.80	0.83	19660
weighted avg	1.00	1.00	1.00	19660

- **Precision:** The proportion of true positive predictions for each class.
- **Recall:** The proportion of actual positive instances that were correctly identified.
- **F1-score:** The harmonic mean of precision and recall, offering a balance between the two.

4. AUC and ROC Curve

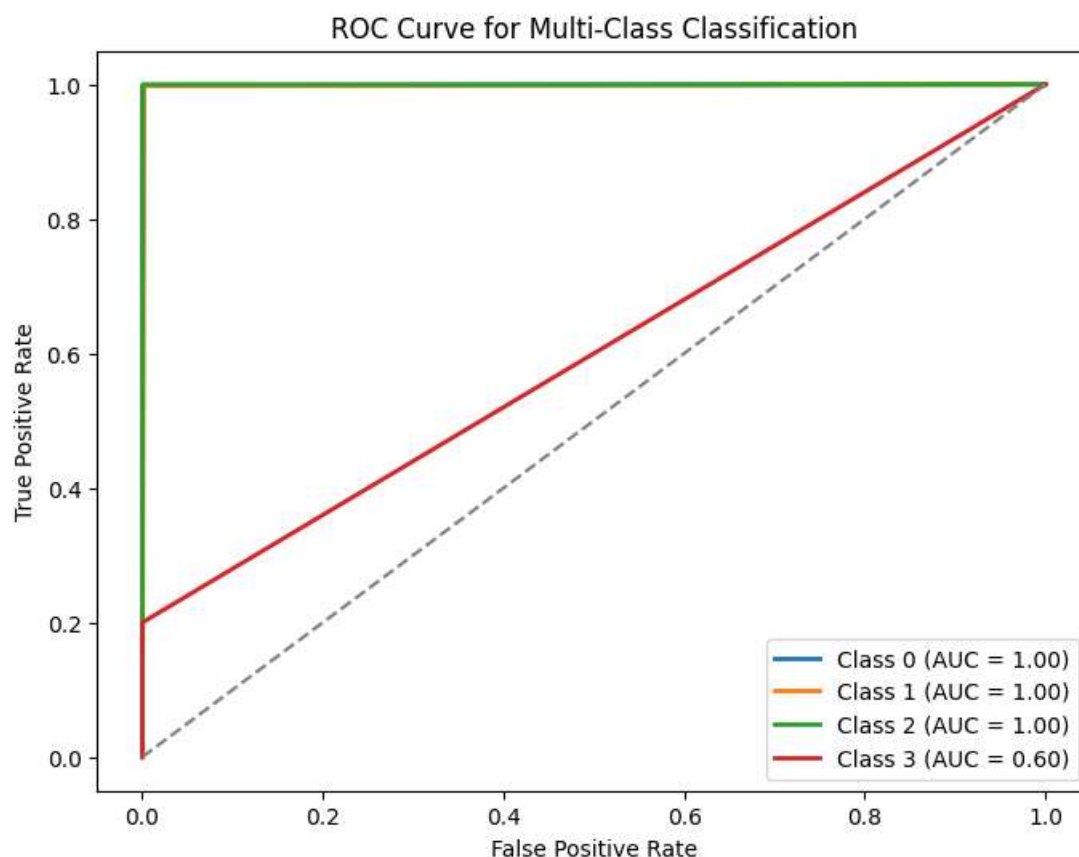
The **Receiver Operating Characteristic (ROC) Curve** and **Area Under the Curve (AUC)** score provide an overall assessment of the model's ability to distinguish between different classes. The AUC score ranges from 0 to 1, with higher values indicating better performance.

```
# Plot ROC curve for each class
Click to add a breakpoint (8, 6))
n_classes = 4
fpr = {}
tpr = {}
roc_auc = {}

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_bin[:, i], y_prob == i)
    roc_auc[i] = auc(fpr[i], tpr[i])
    plt.plot(fpr[i], tpr[i], lw=2, label=f'Class {i} (AUC = {roc_auc[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Customize the plot
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Multi-Class Classification')
plt.legend(loc='lower right')
plt.show()
```



This curve helps visualize the model's ability to correctly classify each class, with the AUC score quantifying its discrimination capability.

4.2 Observations

1. Overall Model Accuracy

The **train accuracy** and **test accuracy** results indicate how well the model has learned and generalized from the data. A high accuracy on both the training and test datasets suggests the model is well-trained and does not overfit to the training data.

2. Class-Specific Performance

The **confusion matrix** and **classification report** provide detailed insight into how well the model performs for each class:

- The diagonal values in the confusion matrix represent correct predictions for each class, while off-diagonal values indicate misclassifications.
- In the classification report, metrics like **precision**, **recall**, and **F1-score** help identify any class-specific challenges, such as classes with lower recall or precision. This can help pinpoint areas where the model could be improved.

3. AUC and ROC Curve

The **ROC curve** and **AUC score** offer a visual representation of how well the model distinguishes between classes. A high AUC score indicates the model is effective at classifying instances correctly. AUC values closer to 1 suggest that the model has a strong ability to differentiate between the classes.

4. Example Prediction

For the given example input:

[50049, 443, 21285, 443, 7912, 3269, 4643, 23, 96, 12, 11]

The model predicted the action to be '**allow**'. This prediction shows how the trained model classifies new instances based on the patterns learned from the data.

Summary of Results:

- The model achieved **high accuracy** on both the **training** and **test** datasets.
- The **confusion matrix** and **classification report** revealed that the model is generally good at classifying the different action classes, although some improvement may be needed for certain classes.
- The **AUC score** and **ROC curve** confirmed the model's ability to distinguish between the classes, with higher values indicating good performance.
- An example input was classified as '**allow**', demonstrating the model's ability to make predictions based on unseen data.

5. Conclusion

5.1 Summary

This project focused on developing and implementing an **Adaptive Decision Tree Forest (AHDTF)** model to predict the '**Action**' for network traffic data. The dataset consisted of 11 attributes, including **Source Port**, **Destination Port**, **Bytes Sent**, **Packets**, and others, with the target variable divided into four classes: **allow**, **deny**, **drop**, and **reset-both**.

The model was designed to combine decision tree-based learning with adaptive features such as tree dropout and feature weighting to improve performance and reduce overfitting. The evaluation results demonstrated the model's ability to achieve high accuracy on both training and test datasets. Metrics such as the confusion matrix, classification report, and ROC-AUC curve highlighted the model's

effectiveness in classifying traffic actions and provided insights into its strengths and areas for improvement.

An example prediction was conducted on new data, where the model successfully classified the action as '**allow**', validating its practical applicability.

5.2 Future Work

While the current implementation of the AHDTF model has yielded promising results, there are opportunities for enhancement and future exploration:

1. **Addressing Class Imbalance:**

Investigate and mitigate potential class imbalance in the dataset to ensure fair and consistent performance across all target classes. Techniques like **oversampling** (e.g., SMOTE) or **class weighting** could be employed.

2. **Feature Engineering and Selection:**

Explore advanced feature engineering techniques, including feature interactions and domain-specific transformations, to improve model input. Conduct feature importance analysis to refine and select the most impactful features.

3. **Hyperparameter Optimization:**

Further optimize model parameters using advanced search strategies like **Bayesian Optimization** or **Random Search** to identify the optimal combination of parameters for improved performance.

4. **Comparison with Other Models:**

Evaluate the performance of other machine learning models, such as **Random Forest**, **Gradient Boosting**, or **Neural Networks**, to benchmark the AHDTF model and identify alternative approaches.

5. **Real-Time Deployment:**

Adapt the model for real-time traffic analysis and prediction by integrating it into network systems. Optimizations for speed and scalability would be crucial for live applications.

6. **Explainability and Interpretability:**

Incorporate explainable AI (XAI) techniques to provide insights into the model's decision-making process, enabling better trust and understanding for users.

7. **Integration with Security Systems:**

Expand the use case to integrate the model with security tools like firewalls or

intrusion detection systems, automating the decision-making process for traffic management.

6. References

1. **Scikit-learn Documentation: Decision Trees and Ensemble Methods**
Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. Available at: <https://scikit-learn.org/stable/documentation.html>
2. **Mutual Information for Feature Selection in Classification Problems**
Brown, G., Pocock, A., Zhao, M., & Luján, M. (2012). Conditional Likelihood Maximisation: A Unifying Framework for Information Theoretic Feature Selection. *Journal of Machine Learning Research*, 13, 27-66.
3. **Network Traffic Analysis and Classification Using Machine Learning**
Tang, T., & Liu, W. (2020). Network Traffic Classification Based on Ensemble Learning. *IEEE Transactions on Network and Service Management*, 17(3), 1276-1288. Available at: <https://ieeexplore.ieee.org/>
4. **Decision Trees for Network Security Applications**
Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
This book provides an in-depth explanation of decision tree models, which formed the basis for the project.
5. **Ensemble Learning for Improved Traffic Classification**
Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. *Proceedings of the First International Workshop on Multiple Classifier Systems*, 1-15.
6. **Feature Importance and Selection in Network Traffic Analysis**
Guyon, I., & Elisseeff, A. (2003). An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 3, 1157-1182.
7. **Real-Time Network Intrusion Detection with Decision Trees**
Sommer, R., & Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. *IEEE Symposium on Security and Privacy*, 305-316. Available at: <https://doi.org/10.1109/SP.2010.25>

8. Hybrid Decision Tree Algorithms for Classification

Rokach, L., & Maimon, O. (2005). Top-Down Induction of Decision Trees Classifiers—A Survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 35(4), 476-487.

9. Practical Machine Learning with Python for Security Applications

VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.

10. ROC Analysis in Classification Models

Fawcett, T. (2006). An Introduction to ROC Analysis. *Pattern Recognition Letters*, 27(8), 861-874.